```
!pip install -q transformers torch accelerate pandas
```

```
import torch
import pandas as pd
import re
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```
file_path = "/content/test_prompts.csv"
data = pd.read_csv(file_path)
```

```
print("Total samples:", len(data))
print("Columns:", data.columns)
print(data.head())
```

```
Total samples: 1319
Columns: Index(['question', 'answer', 'numeric_answer', 'CoT_anot1', 'CoT_anot2',
       'CoT_paths', 'CoT_directions'],
      dtype='object')
                                            question  \
0  Janet’s ducks lay 16 eggs per day. She eats th...
1  A robe takes 2 bolts of blue fiber and half th...
2  Josh decides to try flipping a house.  He buys...
3  James decides to run 3 sprints 3 times a week....
4  Every day, Wendi feeds each of her chickens th...

                                              answer numeric_answer  \
0  Janet sells 16 - 3 - 4 = <<16-3-4=9>>9 duck eg...              18
1  It takes 2/2=<<2/2=1>>1 bolt of white fiber\nS...               3
2  The cost of the house and repairs came out to ...           70000
3  He sprints 3*3=<<3*3=9>>9 times\nSo he runs 9*...             540
4  If each chicken eats 3 cups of feed per day, t...              20

                                           CoT_anot1  \
0  Q: There are 15 trees in the grove. Grove work...
1  Q: There are 15 trees in the grove. Grove work...
2  Q: There are 15 trees in the grove. Grove work...
3  Q: There are 15 trees in the grove. Grove work...
4  Q: There are 15 trees in the grove. Grove work...

                                           CoT_anot2  \
0  Q: Shawn has five toys. For Christmas, he got ...
1  Q: Shawn has five toys. For Christmas, he got ...
2  Q: Shawn has five toys. For Christmas, he got ...
3  Q: Shawn has five toys. For Christmas, he got ...
4  Q: Shawn has five toys. For Christmas, he got ...

                                           CoT_paths  \
0  Q: Betty is saving money for a new wallet whic...
1  Q: Betty is saving money for a new wallet whic...
2  Q: Betty is saving money for a new wallet whic...
3  Q: Betty is saving money for a new wallet whic...
4  Q: Betty is saving money for a new wallet whic...

                                      CoT_directions
0  Use mathemathical operations and write step by...
1  Use mathemathical operations and write step by...
2  Use mathemathical operations and write step by...
3  Use mathemathical operations and write step by...
4  Use mathemathical operations and write step by...
```

```
possible_q_cols = ["question", "Question", "prompt", "input"]
possible_a_cols = ["answer", "Answer", "output", "label"]

question_col = None
answer_col = None

for col in possible_q_cols:
    if col in data.columns:
        question_col = col
        break

for col in possible_a_cols:
    if col in data.columns:
        answer_col = col
        break
```

```
if question_col is None:
    raise ValueError("Question column not found in dataset!")

if answer_col is None:
    print("⚠ Answer column not found. Accuracy cannot be calculated.")

print("Using Question Column:", question_col)
print("Using Answer Column:", answer_col)
```

```
Using Question Column: question
Using Answer Column: answer
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cuda
```

```
model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    low_cpu_mem_usage=True
).to(device)

model.eval()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/token
You will be able to reuse this secret in all of your notebooks.
```

```
def build_prompt(question, mode="zero_shot", exemplars=None):

    if mode == "zero_shot":
        return f"""Solve the following problem.
Give only the final answer.

Question: {question}
Answer:"""

    elif mode == "few_shot":
        prompt = "Here are some solved examples:\n\n"
        for q, a in exemplars:
            prompt += f"Q: {q}\nA: {a}\n\n"

        prompt += f"""Now solve this:

Q: {question}
A:"""
        return prompt

    elif mode == "cot":
        return f"""Solve the problem step by step.
After reasoning, give final answer as:
Answer: <number>

Question: {question}

Let's think step by step."""

    else:
        raise ValueError("Invalid mode")
```

```
      (act_fn): SiLUActivation()
```

```
def generate_answer(prompt, max_new_tokens=128):

    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
def extract_last_number(text):
    numbers = re.findall(r"-?\d+\.?\d*", text)
    return numbers[-1] if numbers else None
```

```
def get_few_shot_examples(k=3):
    examples = []
    for i in range(min(k, len(data))):
        q = str(data.iloc[i][question_col])
        if answer_col:
            a = str(data.iloc[i][answer_col])
        else:
            a = "Example answer"
        examples.append((q, a))
    return examples

few_shot_examples = get_few_shot_examples(k=3)
```

```
def evaluate_mode(mode, n_samples=20):

    correct = 0
    hallucinations = 0
```

```
    for i in range(min(n_samples, len(data))):

        question = str(data.iloc[i][question_col])

        if answer_col:
            gt_answer = str(data.iloc[i][answer_col])
        else:
            gt_answer = None

        if mode == "few_shot":
            prompt = build_prompt(question, mode, few_shot_examples)
        else:
            prompt = build_prompt(question, mode)

        output = generate_answer(prompt)

        if gt_answer:
            pred = extract_last_number(output)
            gt = extract_last_number(gt_answer)

            if pred is None:
                hallucinations += 1
            elif pred == gt:
                correct += 1

        if i % 5 == 0:
            print(f"[{mode}] Processed {i+1}")

    if gt_answer:
        accuracy = correct / n_samples
        hallucination_rate = hallucinations / n_samples

        print("\n------------------------------")
        print("Mode:", mode)
        print("Accuracy:", round(accuracy, 3))
        print("Hallucination Rate:", round(hallucination_rate, 3))
        print("------------------------------\n")

        return accuracy, hallucination_rate
    else:
        print("No ground truth available for accuracy calculation.")
        return None, None
```

```
acc_zero, hall_zero = evaluate_mode("zero_shot")
acc_few, hall_few = evaluate_mode("few_shot")
acc_cot, hall_cot = evaluate_mode("cot")
```

```
[zero_shot] Processed 1
[zero_shot] Processed 6
[zero_shot] Processed 11
[zero_shot] Processed 16

------------------------------
Mode: zero_shot
Accuracy: 0.0
Hallucination Rate: 0.0
------------------------------

[few_shot] Processed 1
[few_shot] Processed 6
[few_shot] Processed 11
[few_shot] Processed 16

------------------------------
Mode: few_shot
Accuracy: 0.0
Hallucination Rate: 0.0
------------------------------

[cot] Processed 1
[cot] Processed 6
[cot] Processed 11
[cot] Processed 16

------------------------------
Mode: cot
Accuracy: 0.05
Hallucination Rate: 0.0
------------------------------
```